

047 独立实验：patch、注意力与预算

先完成043与053；若尚未学习注意力，先走049→052→053再回来。编号不是学习顺序。

1 环境与材料

按environment.yml建立独立环境，实际核验Linux CPU、Python3.12.14、torch2.7.1+cpu、NumPy2.3.5。不下载预训练权重，不调用API。脚本使用原创8×8条形图，训练图池128、验证64、测试256。

```
conda env create -f environment.yml
conda activate dl047
python test_experiment.py
python -O test_experiment.py
python experiment.py --output rerun
python -O experiment.py --output rerun-optimized
```

默认输入跟随脚本位置，显式相对输出按当前工作目录解释。新实验另写目录，保留原输出。GPU、跨平台全新Anaconda安装和浏览器Jupyter UI不在本次实际核验范围。

2 逐窗口证明patch顺序

先构造每个像素各不相同的多通道4×6输入，P=2。手列第0、1、2、3个patch坐标，区分patch位置顺序和patch内部的通道/行/列顺序。自己的实现必须与逐窗口切片一致，而不只与另一个reshape写法相同。

```
import numpy as np
from experiment import patchify, unpatchify
x = np.arange(3 * 4 * 6.).reshape(1, 3, 4, 6)
tokens = patchify(x, 2)
print(tokens.shape, tokens[0, 0])
np.testing.assert_array_equal(unpatchify(tokens, 2, x.shape), x)
```

预期shape是(1,6,12)。第0块包含各通道左上2×2的值，顺序应为0、1、6、7，然后24、25、30、31，最后48、49、54、55。若直接reshape得到0、1、2、3等连续数，说明块内坐标错了。

再取共享矩阵W为12×5、偏置5，比较tokens@W+b与Conv2d(kernel_size=2,stride=2)；核权重为W.T reshape成5×3×2×2。输出展平空间后应逐元素一致。

3 位置排列的三个不同操作

在相同QKV投影下，测试：没有位置向量时重排输入token；有位置向量但只重排内容；位置与内容一起重排。前者输出相应重排且均值不变，中间通常改变均值，最后仍只是重新编号。

写出你比较的对象：是token级等变，还是池化后不变？不要因为最终分类不变就断言每个位置的输出没有变化。讲义探针固定置换[2,0,3,1]，保留具体数值和浮点容差。

输入一像素平移通常不是patch重排。可以用一个非零像素跨越 2×2 块边界的例子验证，解释043中的步幅相位为何仍然相关。

4 完整九参数账本

用讲义两图，各行是一个 1×2 patch，标签1和0。记录每个嵌入e、Q、K、V、分数S、逐行softmax A、上下文z、均值m、预测、残差与两例平均损失。

反向依次填写预测上游r/2、均值的1/2、A/V两支路、softmax的整行雅可比、Q/K两支路，最后把三个e路径相加。分别累计w0/w1与p0/p1，注意前者跨位置共享、后者每个位置独立。

九参数都同时更新，再从原始两图重做下一次前向。第一例损失下降而第二例变大；解释共享参数的冲突而不是删掉第二例。精确中间数保存在results.hand中，本例含指数，使用float64近似，不声称有理数精确值。

这个子模型没有LN、FFN、残差，正式D=8网络才有完整结构。请说明D=1普通LayerNorm为何会把内容变化消掉，以此避免把子模型误当成完整ViT。

5 完整模型的独立数值参照

test_experiment.py用三种不同证据覆盖正式模型：自写多头与官方MultiheadAttention复制同一参数后对齐；CNN/ViT/混合的完整前向由独立NumPy实现；小ViT全部802参数对NumPy标量NLL作中心差分。

这包括patch、位置、残差、LayerNorm、GELU、均值头等真实路径，不只是测试一个孤立softmax。另有九参数账本的全部输入与参数梯度，以及softmax行雅可比与数据完整性检查。测试不使用会被-O删掉的Python assert作最终验收。

检查不代表普遍证明。若自己扩展多头维度、输入分辨率或mask，需要更新相应公式与测试；本模型固定 $8 \times 8/16$ 位置，不会自动插值位置表。核心patch API对不整除尺寸明确报错，不允许把被丢失像素隐瞒掉。

6 冻结的18次比较

训练规模32与128，小集合为同一训练池的前32图。模型CNN、小ViT、混合各3种子4711、4712、4713；每个组合200次全批次SGD，学习率0.15，无动量、权重衰减、早停、增强、预训练或蒸馏。

正式比较每个规模内匹配图数和更新数，未匹配参数数、FLOPs、墙钟或最优调参。跨规模时全批次图呈现量增加4倍，必须报告，不能称为纯固定暴露量的数据效率实验。

输出应有18次拟合、3600更新、3618参数状态、18组最终权重和全部测试logit/每图损失。
training_traces.json.gz包含每一步参数、训练NLL/准确率；final_states.json.gz保存可按state_dict加载的最终张量。压缩包与解压原文摘要都记录在results.json。

所有初始/较差种子都保留。验证集只提供最终描述性读数，没有用来选择200这个步数或学习率；如果开始改配方，另立协议和真正的验证选择/测试封存边界。

7 解读失败与事后诊断

32图时CNN、小ViT、混合测试准确率均值约88.80%、63.93%、71.48%；128图时约90.49%、52.73%、90.76%。先核对这些固定数值，再解释什么结论不能由它们得出。

小ViT没有随数据增加而改善；CNN准确率高但NLL约0.687，接近log2。看logit间距图，区分预测方向与置信度。不要挑一个架构最好的种子，或把小预算结果写成所有视觉Transformer的排名。

神经结果出来后才加入一个非学习方向相关基线：水平相邻乘积和减竖直相邻乘积和，分数 ≥ 0 判横条。它在本数据上100%，明确属于事后诊断，利用任务结构，没有拟合参数或调整神经协议；它不是原先冻结的第四个训练处理。分数未校准为概率，不报NLL。

讨论它能支持什么：任务存在简单可行规则，复杂网络当前表现并不等于任务本身无法预测。它不能证明哪一种训练设置是失败的唯一原因。提出一个新的受控诊断计划即可，不要在同一测试集上无限试到“好结果”。

8 资源与显存边界

先手算 $B=32$ 、 $h=2$ 、 $N=16$ 、float64的单个注意力矩阵容量：131072字节。再阅读resource_audit中CPU实际保存Tensor的逻辑字节和去重storage字节，解释为什么它们含输入/参数引用且不是总进程峰值。

CUDA在本次环境不可用，显存峰值明确未测。若你自行在GPU补做，需要记录设备型号、dtype、优化器状态、批次、训练/推理模式、预热、同步与峰值重置。比较max_memory_allocated与reserved时区分含义，不把CPU统计数填进显存栏。

改变patch边长时，先推导N再算 BhN^2 。图像长宽翻倍、P固定时矩阵容量16倍；P减半也16倍。

FlashAttention等不同后端可能不显式保存整矩阵，不要把本文手写稠密实现的估计当作所有实现的实测。

9 Notebook、图与提交

Notebook从单元或仓库根启动，核对源码/数据/结果/图摘要，然后在临时目录重新运行全部18拟合并对齐结果和轨迹。实际输出图由本轮数据绘制，采用显式PNG显示，避免进程内核plt.show没有保存图。

```
python make_figures.py --output rerun-figures
```

绘图需要Noto Sans CJK，可设置DL_CJK_FONT。已交PDF直接可读；重建PDF看shared/build-tools。所有生成器均以原创数据为输入，不涉及第三方模型许可或下载。

提交：patch与卷积等价核验、完整九参数账本、位置三项对照、18次完整比较、预算/资源表、事后基线身份说明，以及不能推出的结论。建议评分：机制与shape25分、反向账本25分、独立核验20分、公平比较及失败解释20分、可重跑证据10分。